GROUP ASSIGNMENT 2 (WEEK 7) – IF260 Operating System

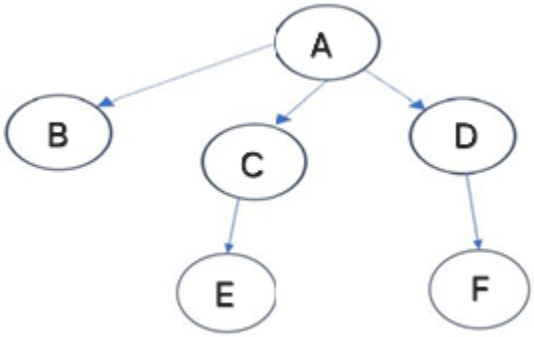
Group : 4

Name (NIM) :

- Steven Santoso (00000138789)
- Ang Farrel Kurnia (00000141596)
- Dustin Nathanael Bernadi (00000141880)
- Abraham Immanuel Soerjana (00000140413)

1. **Based on the code snippet above, answer the following questions:**

- **Assuming the code above is run in a Linux environment, explain whether a zombie process occurs, and in which part (which line) of the code it occurs.**
- Zombie process terjadi karena child keluar sebelum parent membaca exit status child sehingga child akan mengalami zombie process. Hal ini terjadi karena saat pid membelah menjadi parent dan child menggunakan fork(), child akan memiliki pid yang bernilai 0. Setelah proses child, parent akan menunggu selama 10 detik.
  Namun, yang terjadi adalah child menjadi zombie karena parent tidak menunggu status exit child dengan waitpid. Jika parent exit sebelum child selesai, maka child akan menjadi orphan. Zombie process terjadi di line kondisi if (pid > 0).
- **Which part of the code needs to be modified so that after the child process is finished, the parent process immediately knows and prevents the zombie process from occurring.**
- Kode pada kondisi if perlu ditambahkan kode berikut:
  waitpid (pid, NULL, 0);
  sebelum line printf("Let's wait for 10 seconds before exiting").
- **Modify the code above so that it will produce the following process hierarchy. Show your program output that validates this hierarchy**



```
PID b = 2889, PPID = 2888
b finish...

PID e = 2892, PPID = 2890
e finish...

PID f = 2893, PPID = 2891
f finish...

PID d = 2891, PPID = 2888
d finish...

PID c = 2890, PPID = 2888
c finish...

PID a = 2888, PPID = 2243
a finish...
```

2. **<u>Question:</u>**

- **<u>What is the purpose of the pthread_create() function?</u>**
- Fungsi dari pthread_create() merupakan syntax untuk membuat thread baru
- **<u>Why is it necessary to use two pthread_join() functions before the main program completes?</u>**
- Karena untuk menunggu thread1 dan thread2 menyelesaikan tugasnya, agar program tidak terminate dahulu sebelum thread1 dan thread2 belum sempat jalan. Mengapa dibutuhkan nya dua pthread_join() karena sifat dari fungsi pthread_join() sendiri bersifat 1 to 1, yang dimana satu pthread_join() hanya untuk satu thread spesifik hingga selesai.
- **<u>Explain how the print_message_function() function is called by each thread?</u>**
- Dalam pthread_create(), parameter nya memanggil function print_message_function() yang menjadi fungsi yang akan dijalankan oleh thread
- **<u>Is the output of this program predictable each time the program is run? Explain your answer.</u>**
- Tidak, program ini tidak menjamin menghasilkan output yang sama setiap kali dijalankan. Karena kedua thread memiliki fungsi yang sama, terutama keduanya memanggil function “print_message_function” yang didalamnya terdapat “sleep(1);” yang menyebabkan kedua thread akan tidur selama 1 detik, sehingga kedua thread akan bangun secara bersamaan dan berlomba merebut perintah printf. Mengapa outputnya setiap dijalankan bisa berbeda? terdapat urutan eksekusi (scheduler OS) yang bersifat

non-deterministik, yang berarti keputusan eksekusi bergantung pada kondisi sistem pada saat itu, sehingga outputnya berbeda-beda setiap kali dijalankan.

- **Modify the program by placing pthread_join() correctly, so that thread2 always prints a message after thread1.**
- Jadi penempatan pthread_join() yang benar agar thread 2 selalu print setelah thread 1 adalah dengan memindahkan "pthread_join (thread1, NULL);" sebelum thread 2 berjalan.

```
int main ()
{
pthread_t thread1, thread2;
const char *message1 = "Thread 1";
const char *message2 = "Thread 2";
int iret1, iret2;

iret1 = pthread_create(&thread1, NULL, print_message_function,(void*)
message1);

if (iret1) {
fprintf (stderr,"error - pthread_create() return code: %d\n", iret1)
;
exit(EXIT_FAILURE);
}
pthread_join (thread1, NULL); //Ditempatkan disini

iret2 = pthread_create (&thread2, NULL, print_message_function,
(void*) message2);

if (iret2) {
fprintf (stderr, "Error - pthread_create() return code: %d\n", iret2)
;
exit (EXIT_FAILURE);
}

pthread_join (thread2, NULL) ;
exit(EXIT_SUCCESS) ;
}
```

3. **Question:**

- **Explain how synchronization is done to accommodate the activities of a barber and customers coming to this barbershop?**

  Sinkronisasi dilakukan menggunakan semaphore untuk mengatur antrean dan status:

Semaphore	Nilai Awal	Peran
waitingRoom	numChairs	Membatasi jumlah customer di ruang tunggu
barberChair	1	Memastikan hanya ada 1 customer di kursi barber
barberPillow	0	Memberikan sinyal untuk membangunkan barber dari customer
seatBelt	0	Memberikan sinyal selesai memotong dari barber ke customer

- **Convert this sleeping-barber problem into multi-processes with semaphores for communication and synchronization. One process is assigned for a barber, and one process is assigned for customers that run as multi-threads. You can create two separate .c files for barber and customers, or use one .c file and create the additional process using the fork system call.**

  barber.c

4. **Question:**

- **Determine the timeline and average turnaround time (the average time from the 3 process's arrival to its completion) using the following scheduling algorithm:**
  - **First-Come First-Served**

    Gantt chart

A	B	C	D	E	F

0 5 8 10 15 17 18

Turn Around Time : Completion Time - Arrival Time

: A = 5 - 0 = 5
B = 8 - 1 = 7
C = 15 - 2 = 8
D = 15 - 5 = 10
E = 17 - 7 = 10
F = 18 - 8 = 10

Average Turn Around Time : $\frac{5+7+8+10+10+10}{6} = \frac{50}{6} = 8,33$

  - **Shortest Job First**

    Gantt chart

A	C	E	F	B	D

0 5 7 9 10 13 18

Turn Around Time : Completion Time - Arrival Time

: A = 5 - 0 = 5
B = 13 - 1 = 12
C = 7 - 2 = 5
D = 18 - 5 = 13
E = 9 - 7 = 2
F = 10 - 8 = 2

Average Turn Around Time : $\frac{5+12+5+13+2+2}{6} = \frac{39}{6} = 6,5$

- **<u>Shortest Remaining Time Next.</u>**

  Gantt chart

A	B	C	A	E	F	A	D

0 1 4 6 7 9 10 13 18

Turn Around Time : Completion Time - Arrival Time

: A = 13 - 0 = 13
B = 4 - 1 = 3
C = 6 - 2 = 4
D = 18 - 5 = 13
E = 9 - 7 = 2
F = 10 - 8 = 2

Average Turn Around Time : $\frac{13+3+4+13+2+2}{6} = \frac{37}{6} = 6,17$

- **<u>Round-Robin with 1 unit quantum time</u>**

  <u>Gantt chart</u>

A	B	C	D	E	F	A	B	C	D	E	A	B	D	A	D	A	D

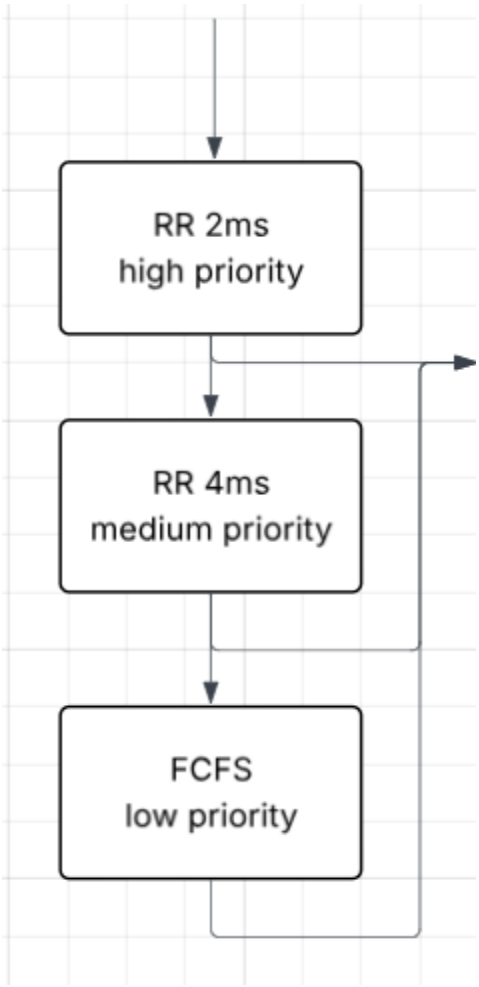
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18

Turn Around Time : Completion Time - Arrival Time

: A = 17 - 0 = 17
B = 13 - 1 = 12
C = 9 - 2 = 7
D = 18 - 5 = 13
E = 11 - 7 = 4
F = 16 - 8 = -2

Average Turn Around Time : $\frac{17+12+7+13+4-2}{6} = \frac{51}{6} = 8,5$

- **<u>If a multi-level feedback scheduling algorithm is used, sketch the number of levels and the scheduling method used at each level. It is assumed that priority levels are influenced by the target run time.</u>**



<u>Sistem ini menggunakan 3 antrean di mana setiap proses baru masuk ke Q1 (RR 2ms). Jika melebihi kuantum, proses di-demote ke Q2 (RR 4ms), dan akhirnya ke Q3 (FCFS). Untuk menghindari starvation, dilakukan priority boost secara berkala.</u>

5. **Question:**

- **Explain the characteristics of deadlock that may occur in the code above!**

Deadlock dalam kode tersebut terjadi karena terpenuhinya kondisi Circular Wait. Karakteristiknya adalah sebagai berikut:

- **Mutual Exclusion** : Masing-masing mutex (lock1 dan lock2) hanya dapat dikuasai oleh satu thread pada satu waktu

- **Hold and Wait** : Thread 1 telah memegang lock1 dan menunggu lock2, sementara Thread 2 telah memegang lock2 dan menunggu lock1

- **No Preemption** : Mutex tidak dapat diambil paksa dari thread yang sedang memegangnya sebelum thread tersebut melepaskannya secara sukarela

- **Circular Wait** : Terjadi rantai melingkar di mana Thread 1 menunggu resource yang dipegang Thread 2, dan Thread 2 menunggu resource yang dipegang Thread 1, sehingga kedua thread berhenti selamanya

- **Provide a strategy to avoid deadlock in the code example above!**

Strategi yang paling efektif untuk kode ini adalah menggunakan Resource Ordering atau Lock Ordering. Caranya adalah dengan menetapkan urutan penguncian yang sama untuk semua thread. Dalam contoh ini, Thread 2 harus diubah agar meminta kunci dengan urutan yang sama seperti Thread 1, yaitu mengambil lock1 terlebih dahulu baru kemudian lock2. Dengan urutan yang konsisten, kondisi circular wait tidak akan pernah terbentuk

- **Change the mutex mechanism in the program by using semaphores! What is the difference between the working mechanism of a semaphore and a pthread mutex?**

Dalam mutex, resource bersifat eksklusif sehingga hanya thread yang melakukan lock yang bisa melakukan unlock. Jika Thread 1 macet saat memegang lock1, resource tersebut terkunci selamanya. Sedangkan pada Semaphore, mekanisme bekerja berdasarkan sinyal (counter). Sem_wait mengurangi nilai izin, dan sem_post menambahnya. Karena tidak ada konsep "pemilik", thread lain bisa mengirim sinyal post untuk membebaskan resource yang tertahan, memberikan fleksibilitas lebih dalam sinkronisasi antar-proses.